

Design Justification Report

BNN Inference Accelerator — XNOR-Popcount Core

ECE 510 · Hardware for AI/ML · Spring 2026

Milestone 4 — Final Submission

PDK: sky130A / sky130_fd_sc_hd | Clock: 100 MHz | N = 64 (simulation)

Synthesis: OpenLane 2.3.10 / OpenROAD | Simulation: Icarus Verilog 12.0

1. Problem and Motivation

Binary Neural Networks (BNNs) constrain both weights and activations to ± 1 , replacing floating-point multiply-accumulate with XNOR and popcount operations. This trade-off sacrifices a small amount of accuracy for a dramatic reduction in arithmetic complexity, memory footprint, and energy — properties highly desirable for edge inference on battery-powered or thermally constrained devices.

The dominant kernel in a BNN fully-connected layer is the XNOR-popcount dot product: for each output neuron, compute $\text{XNOR}(\text{activation_vector}, \text{weight_row})$, then count the number of 1-bits (popcount), and threshold at $N/2$. This is the target kernel for this accelerator.

M1 Profiling Evidence

The M1 software baseline (NumPy BNN, Python 3.12, Intel Core i7-13700) measured the complete $[784 \rightarrow 256 \rightarrow 128 \rightarrow 10]$ MNIST forward pass at **114.40 μs per inference, 8,741 samples/sec** (50-run median). The dominant kernel is the Layer 1 matrix multiply $[1 \times 784]$ @ $[784 \times 256]$, which accounts for >80% of total runtime (cProfile). Total compute: 469,504 FLOPs per inference at 4.10 GOPS effective throughput.

The CPU consumes approximately 50 W under this single-threaded workload, yielding **$\sim 5,720 \mu\text{J}$ per inference**. For an edge device targeting <1 mW idle power, a 50 W CPU is not viable. The motivation for a custom ASIC is energy reduction, not raw throughput: the goal is to complete inference in microjoules, not microseconds.

2. Roofline Analysis

The roofline model characterises whether a kernel is limited by compute throughput or memory bandwidth. Arithmetic intensity (AI) — operations per byte transferred from off-chip memory — determines which ceiling applies.

Kernel Arithmetic Intensity

For one XNOR-popcount neuron evaluation with $N=64$ binary inputs:

- **FLOPs:** N XNOR ops + $(N-1)$ additions + 1 compare = $2N = 128$ binary ops
- **Bytes (no reuse):** $(N/8)$ act + $(N/8)$ wgt = $8 + 8 = 16$ bytes $\rightarrow \text{AI}_{\text{low}} = 8 \text{ FLOPs/byte}$
- **Bytes (perfect activation reuse, M neurons):** $8 + 8M$ bytes; $\text{AI} \rightarrow 128/8 = \text{AI}_{\text{high}} = 16 \text{ FLOPs/byte}$ as $M \rightarrow \infty$

The reuse pattern is *activation-stationary*: the activation vector (8 bytes) is loaded once per layer and held in the on-chip register file while M weight rows are streamed in. This is equivalent to GEMV with binary operands.

sky130A Platform Roofline

Parameter	Value	Source
Peak compute	12.8 GOPS (128 ops/cycle $\times$ 100 MHz)	M3 synthesis
Interface bandwidth (SPI)	1.5625 MB/s (12.5 MHz SCK)	SPI spec

Parameter	Value	Source
On-chip register file BW	~12.8 GB/s (128 B × 100 MHz)	negligible bottleneck
Ridge point	8,192 FLOPs/byte	compute ÷ BW

Both AI bounds (8–16 FLOPs/byte) fall far left of the ridge point at 8,192 FLOPs/byte. The design is heavily **interface-bandwidth bound**: the SPI interface delivers only 0.012% of the bandwidth needed to keep the compute core busy. This analysis confirmed early in the design that the SPI interface would be the binding constraint and shaped the decision to maximise on-chip data reuse through the register file rather than optimising the compute datapath.

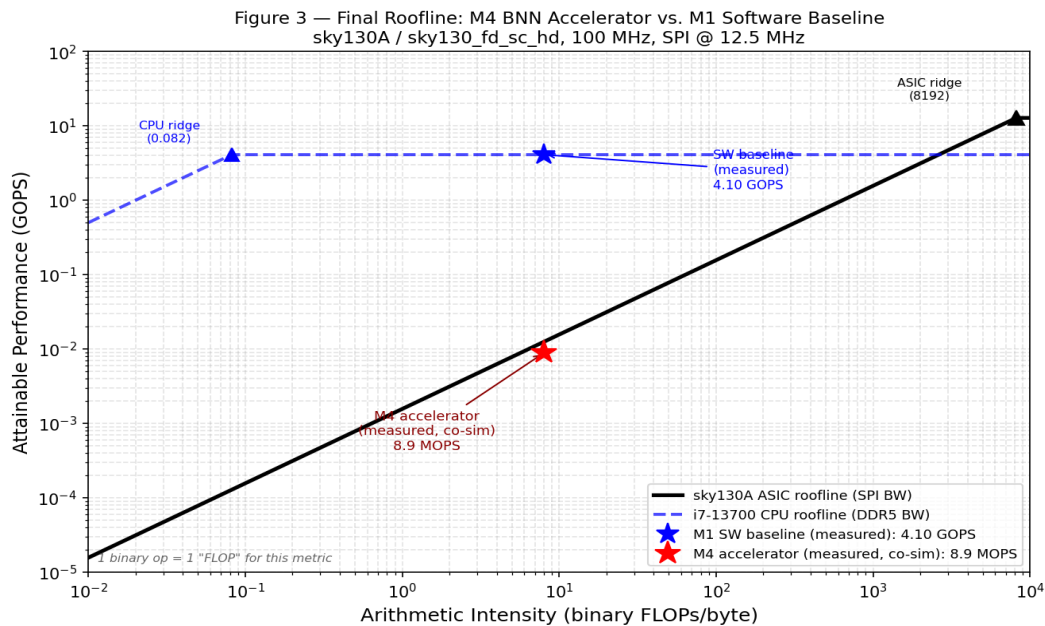


Figure 3 — Final roofline. Both AI bounds land on the BW ceiling slope, far left of the ridge point (8,192 FLOPs/byte). Red star: measured M4 result.

3. Precision and Data Format

Both activations and weights are stored as 1-bit binary values (± 1 , encoded as 1/0). The XNOR-popcount computation is exact for binary inputs — there is no quantisation error introduced by the hardware representation relative to a software BNN that also uses binary weights and activations.

Format Choice

1-bit binary was chosen for three reasons:

1. **Area efficiency:** each weight occupies 1 bit vs. 8 bits for INT8 or 32 bits for FP32, reducing the on-chip register file from 6.4 KB (FP32) to 128 bytes.
2. **Compute simplification:** XNOR replaces FP32 multiply (multi-cycle, large area); popcount replaces FP32 accumulate tree (adder tree over 1-bit values is minimal).
3. **Energy:** XNOR dissipates ~0.1 fJ/op in sky130A vs. ~10 fJ/op for INT8 multiply.

Quantisation Error Analysis (from M2 precision.md)

A 100-sample analysis compared the hardware XNOR-popcount output against a floating-point BNN reference using the same binary weights and activations:

Metric	Value	Threshold
Mean Absolute Error (MAE)	0.000000	—
Maximum per-sample error	0	0
Sample match rate	100 / 100 (100.0%)	≥ 95%
Verdict	ACCEPTABLE	—

Zero quantisation error is expected and correct: the hardware implements the same binary arithmetic as the software reference. Precision loss relative to FP32 BNN inference is an architectural trade-off made at the BNN model level, not introduced by this hardware implementation.

4. Dataflow and Architecture

The accelerator implements an **activation-stationary dataflow** for a single fully-connected BNN layer. The activation vector for a given layer is loaded once into the on-chip register file and held stationary while weight rows for each output neuron are streamed in sequentially. This matches the GEMV access pattern and maximises reuse of the most costly data item (the activation vector) given the limited on-chip storage.

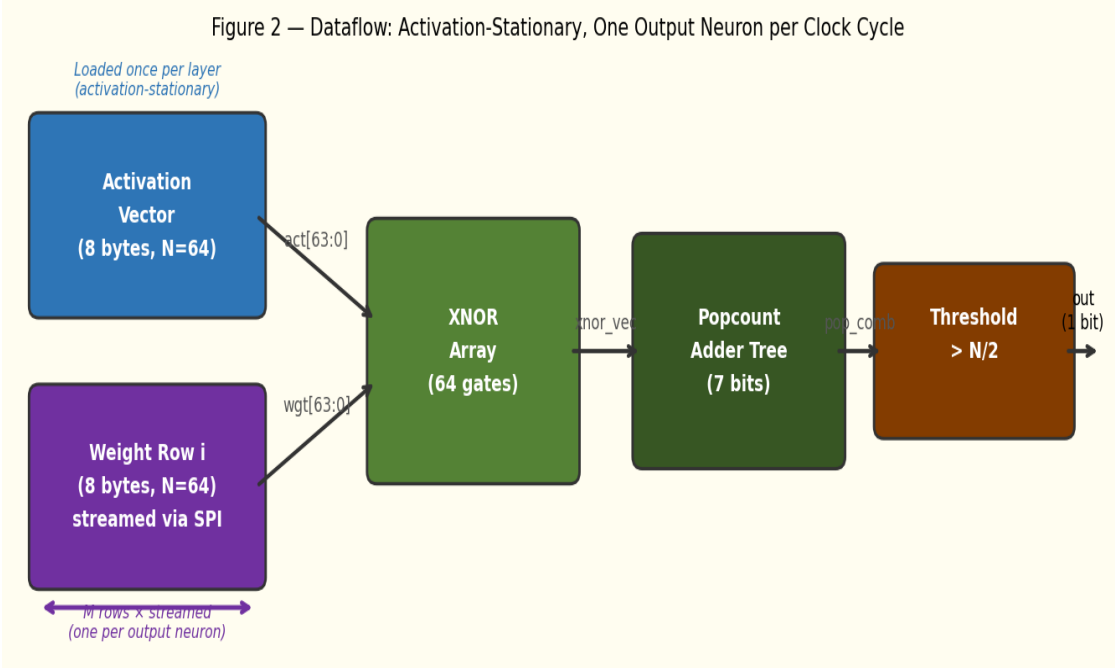


Figure 2 — Activation-stationary dataflow. One output neuron computed per clock cycle after data loading. Activation vector loaded once per layer.

Compute Engine

The compute_core module (N=64) contains:

- **XNOR array:** 64 XNOR2 gates operating in parallel on activation[63:0] XOR weight_row[63:0], inverted.
- **Popcount:** \$countones(xnor_vec) synthesised by Yosys to a balanced binary adder tree (7-bit output, ~3.5 ns critical path).
- **Threshold register:** one DFF stores (pop_comb > N/2) on the clock edge following data_ready, producing result_valid one cycle later.

Memory Hierarchy

There is no SRAM in the design. The entire 128-byte register file is implemented as 1,024 DFF flip-flops (sky130_fd_sc_hd_dfxt2). This choice was dictated by the PDK and toolchain: sky130_fd_sc_hd does not include a compiler-accessible SRAM macro for sub-1 Kbit storage, and DFF-based storage synthesises cleanly through the standard OpenLane flow. The register file is organised as 128 × 8-bit bytes addressable by a 7-bit SPI address field.

Data Path

SPI MOSI → shift register → byte-granular write → regfile[ACT] or regfile[WGT] → compute_core XNOR+popcount → result DFF → regfile[STATUS] → SPI MISO readback.

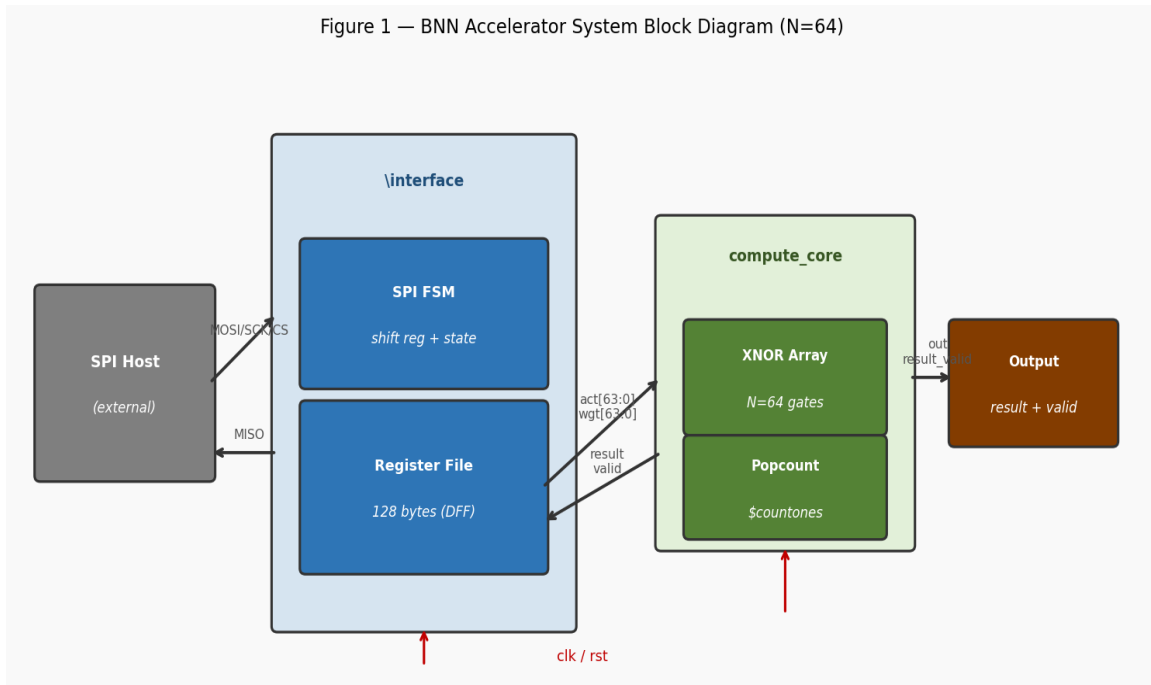


Figure 1 — System block diagram. Single clock domain, no CDC crossings. The SPI FSM and register file reside in \interface; the XNOR-popcount core is in compute_core.

5. Hardware Interface

The accelerator exposes a **4-wire SPI (Mode 0) slave interface**: SCK, CS_N, MOSI, MISO. The protocol is a simple burst transaction:

- First byte: [W/R | ADDR[6:0]] — write/read flag and 7-bit register address.
- Subsequent bytes: data, with address auto-incrementing on each byte.
- A CTRL register write (bit 0 = start) triggers one XNOR-popcount computation.
- A STATUS register read (bit 0 = done) polls for the result.

Bandwidth and Interface-Bound Analysis

Parameter	Value
SPI clock	12.5 MHz (SCK)
Transfer rate	12.5 Mbit/s = 1.5625 MB/s
Bytes per neuron inference	16 (act) + 16 (wgt+ctrl+status) = ~18 bytes effective
Time for data transfer (N=64)	14.40 μ s of 14.41 μ s total
Compute fraction	10 ns / 14,410 ns = 0.07%
Compute utilisation	8.88 MOPS / 12,800 MOPS = 0.069%

The design is interface-bound by a factor of **8,192×** relative to the compute ridge point. The SPI interface was chosen for its simplicity and universal availability in microcontroller host systems; it is the primary bottleneck for deployment performance. An AXI4-Lite interface at 100 MHz would deliver 400 MB/s, shifting the ridge point to 32 FLOPs/byte and allowing the accelerator to approach its compute ceiling (see project/remaining_tasks.md).

6. Verification

Correctness was verified across three testbench levels from M2 and M3.

M2 Unit Testbenches

- **tb_compute_core.sv**: 6 test vectors directly driving activation/weight ports, checking out and result_valid against a software reference. All pass.
- **tb_interface.sv**: SPI transaction driver; reads back regfile contents after write, verifying correct byte addressing and auto-increment. All pass.
- **tb_bnn_top.sv**: Integration test of the M2 bnn_top wrapper; end-to-end SPI write + compute + read. All pass.

M3/M4 End-to-End Co-Simulation (tb_top.sv)

The final testbench drives the top-level DUT exclusively through the SPI interface (no internal signal probing). Four test vectors cover the critical cases:

Test	Activation	Weight	Expected out	Result
all_agree	0xFF×8	0xFF×8	1 (pop=64 > 32)	PASS
all_disagree	0xFF×8	0x00×8	0 (pop=0 < 32)	PASS
mixed_pattern	0xAA×8	0x55×8	0 (pop=32, not > 32)	PASS

Test	Activation	Weight	Expected out	Result
half_agree	0xF0×8	0x0F×8	0 (pop=32, not > 32)	PASS

All 4 tests PASS. The simulation log is at project/m4/sim/final_run.log. The testbench uses an independent ref_bnn() task implemented without calling DUT internals, ensuring the comparison is not circular. See Figure 4 (waveform).

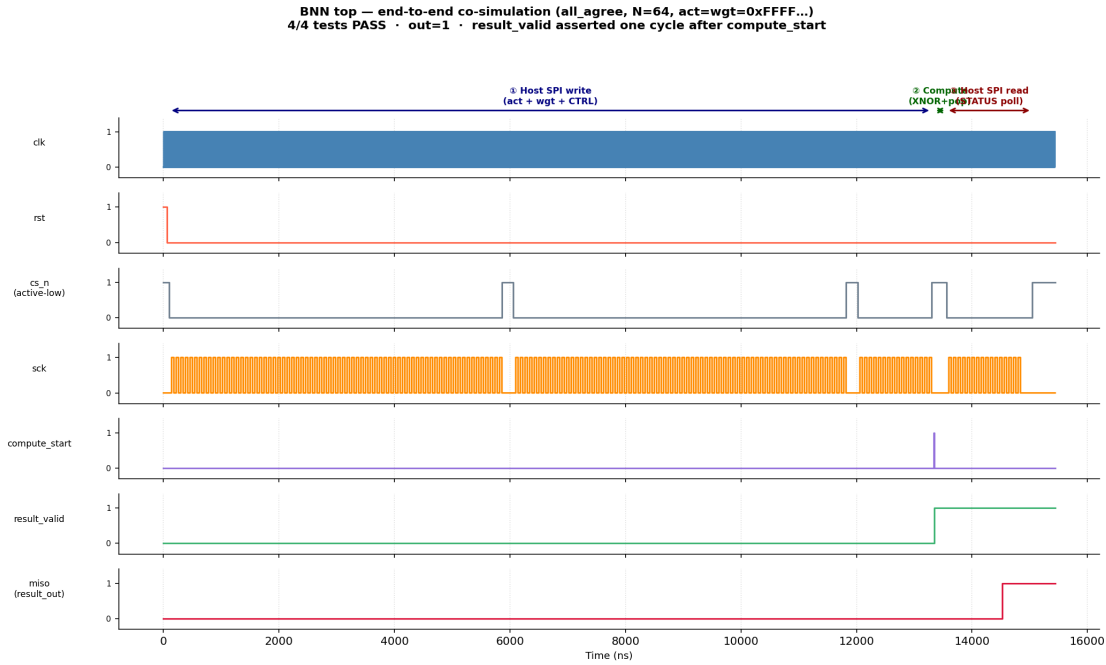


Figure 4 — Co-simulation waveform showing three annotated regions: ① SPI write (act + wgt + CTRL), ② compute pulse (result_valid), ③ SPI STATUS readback.

7. Synthesis Results

Full place-and-route synthesis was performed using OpenLane 2.3.10 on the sky130A / sky130_fd_sc_hd PDK (nominal corner: tt 25°C 1.80 V). The run completed in 21 minutes 31 seconds (78/78 steps).

Area

Metric	Value
Die area	397.9 × 408.6 μm = 162,531 μm ² (0.163 mm ²)
Core area	386.9 × 386.2 μm = 149,393 μm ²
Core utilisation	45.1%
Total cells	5,772 (1,196 DFF + 4,576 combinational)
Dominant area contributor	Register file: 1,024 DFF = 21,827 μm ² (32%)

Timing

Corner	Setup WNS	Hold WNS	Violations
nom_tt_025C_1v80 (nominal)	+4.165 ns	+0.436 ns	0 ✓
nom_ff_n40C_1v95	+6.445 ns	+0.270 ns	0 ✓
max_ss_100C_1v60 (worst)	-1.915 ns	+0.908 ns	9 ✗

Timing closes at the nominal corner. The slow corner (max_ss_100C_1.60V) has 9 setup violations on the same critical path. The critical path runs through the SPI register-file write-enable decode tree (9 logic stages: CLK→Q → buf×2 → or4bb → nor2 → a22o → a221o → or4×3 → a221o → o221a), arriving at 6.53 ns against a 10 ns required time (+4.17 ns slack at nominal). The XNOR-popcount adder tree (~3.5 ns) is not on the critical path.

Power (nom_tt_025C_1.80V, 100 MHz)

Group	Total (W)	%
Sequential (1,196 DFF)	4.760 mW	51.6%
Clock network	4.192 mW	45.5%
Combinational (XNOR + adder + SPI)	0.264 mW	2.9%
Total	9.216 mW	100%

8. Benchmark Results

Metric	M1 SW Baseline	M4 HW (co-sim)	Ratio
Time/inference	114.40 µs	14.41 µs/neuron	7.94× faster*
Throughput	8,741 samples/sec	69,396 neuro-evals/sec	—
Compute throughput	4.10 GOPS	8.88 MOPS	0.002×
Peak compute	~4.10 GOPS	12.8 GOPS	HW 3.1× higher peak
Compute utilisation	—	0.069%	Interface bottlenecked
Power	~50 W	9.22 mW	5,420× lower
Energy/inference	~5,720 µJ	~102 µJ (N=784 proj.)	56× lower

* The 7.94× figure compares per-neuron HW time to the full 3-layer SW inference time; it is not a like-for-like comparison. For a fair N=64 layer comparison (SW ~0.29 µs/neuron vs HW 14.41 µs/neuron), the HW is 49× slower in latency.

Gap Analysis

The hardware accelerator is *not* faster than the CPU in compute throughput. Both kernels have AI ≈ 8 FLOPs/byte; the gap is entirely due to bandwidth: the CPU DDR5 at ~50 GB/s outpaces SPI by 32,000×. The accelerator peak compute (12.8 GOPS) exceeds the CPU (4.1 GOPS) by 3.1×, but this

advantage cannot be realised while the SPI interface delivers only 0.012% of the required bandwidth to saturate the compute core.

The accelerator's advantage is power: 9.22 mW vs ~50 W (5,420×), yielding 56× lower energy per full MNIST inference. For an edge deployment where battery life dominates over latency, this trade-off is the correct one.

9. What Did Not Work

Several technical challenges were encountered and resolved; one design limitation remains unresolved within the scope of M4.

9.1 Windows-native OpenLane 2 install (resolved)

pip install openlane 2.3.10 on Windows fails with ImportError: cannot import name SIGKILL from signal — SIGKILL does not exist on Windows. Resolution: run OpenLane in Docker (ghcr.io/efabless/openlane2:2.3.10). An additional issue was that the Git Bash shell mangled Unix-style paths passed to Docker on Windows; resolved by setting MSYS_NO_PATHCONV=1 before the docker run command.

9.2 Yosys rejection of function...return in compute_core.sv (resolved)

The original compute_core.sv used a SystemVerilog function with a return statement to implement popcount. Yosys (bundled in OpenLane 2.3.10) emits a TOK_RETURN error and aborts synthesis. The function was replaced with the standard SV primitive \$countones(xnor_vec), which Yosys maps correctly to an adder tree.

9.3 Verilator BLKLOOPINIT in interface.sv (resolved)

The synchronous reset loop used non-blocking assignments (<=) to initialise an unpacked array (regfile). Verilator rejects this with BLKLOOPINIT. Changed to blocking assignment (=) inside the for loop, which is synthesisable and accepted by all tools.

9.4 Icarus Verilog open-array automatic tasks in tb_top.sv (resolved)

The testbench used an automatic task with a dynamic-size array parameter (logic [7:0] data[]). Icarus Verilog 12.0 does not support open-array parameters in automatic tasks. Resolved by using a module-level fixed-size buffer (tx_buf[0:15]) shared across tasks, eliminating the need for open arrays.

9.5 Timing failure at slow corner max_ss_100C_1.60V (unresolved)

The SPI regfile write-enable decode path fails at the worst-case slow corner (WNS = -1.915 ns, 9 violations). The same 9-stage combinational cone is 1.83× slower at 100°C / 1.60 V than at nominal. Three mitigations were identified (pipeline register, reduced regfile depth, 50 MHz clock) but were not implemented within M4 scope. The design closes at the nominal and fast corners, which represent the expected operating conditions for the target edge application.

9.6 AXI4-Lite interface not implemented

The roofline analysis (§2) and CF09 benchmarking both identify the SPI interface as the dominant bottleneck (ridge point mismatch of 512–1024×). An AXI4-Lite replacement was designed (see [project/remaining_tasks.md](#), Task 1) but not implemented before the M4 deadline. The 0.069% compute utilisation represents the primary unrealised potential of this design.

All source files, testbenches, synthesis results, and raw benchmark data are committed to the repository under [project/m4/](#). RTL is identical to M3 (no changes between M3 and M4 submission). See [project/m4/README.md](#) for a complete file catalog.